

Praneesh R V
CB.SC.U4CYS23036

SSE - Lab Exercise 6 - Docker & Container Security

Task 1: Core Container Architecture

Q1. Define Container, Image, and Volume.

Image - a read-only template built from an ordered stack of filesystem layers plus a configuration manifest (entrypoint, default command, environment, exposed ports, user). It is the packaged artifact: a build output that is distributed and versioned, and it does not execute anything on its own.

Container - a running (or stopped) instance of an image. Docker takes the image's read-only layers, adds a thin writable layer on top via a union filesystem, and executes the configured process inside a set of Linux namespaces (PID, network, mount, UTS, IPC, user) constrained by cgroups. The container is the process plus its isolated view of the system; the writable layer lives and dies with it.

Volume - a storage location managed by Docker that exists outside the container's union filesystem and is mounted into the container at a chosen path. Its lifecycle is independent of any container: writes to a volume bypass the copy-on-write layer entirely and survive docker rm.

Q2. Which of these three components is immutable (read-only) by design?

The image. Its layers are content-addressed - each is identified by a SHA-256 digest of its contents - so any change produces a different layer and therefore a different image, rather than mutating the existing one. That immutability is what makes an image reproducible, cacheable, and verifiable: the same tag pinned to the same digest yields byte-identical filesystem content on every host that pulls it.

Q3. Why is a volume required for database containers if the container can write to its own writable layer?

The writable layer is bound to the container's lifecycle, not the data's. Three concrete problems make it unusable for a database:

Lifecycle mismatch. docker rm destroys the writable layer. Any routine operation - upgrading Postgres 15 to 16, changing an environment variable, recreating the service after a Compose edit - replaces the container and silently discards the entire database. Performance. Union/copy-on-write filesystems (overlay2) copy a whole file up to the writable layer on first write. A database performing small random writes into large data files hits this copy-up cost repeatedly, which is the pathological case for the storage driver.

Operational access. Data in a writable layer is buried inside Docker's storage driver and cannot be backed up, snapshotted, or bind-mounted to a fast dedicated disk. A volume is a first-class object that can be inspected, backed up, and moved between containers. The design rule this expresses: containers should be disposable, data should not be.

Task 2

Container lifecycle operations

0s ~

• ▶ docker pull nginx:alpine

alpine: Pulling from library/nginx

fb597529c916: Pull complete

1ed8e39a7434: Pull complete

8cdfe4f23778: Pull complete

0d62d88506ba: Pull complete

d94291c26261: Pull complete

0ea935727878: Pull complete

da4dea1b00af: Pull complete

76cd5dde9375: Download complete

43aeb74b2469: Download complete

Digest: sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913

Status: Downloaded newer image for nginx:alpine

docker.io/library/nginx:alpine

What's next:

View a summary of image vulnerabilities and recommendations → [docker scout quickview nginx:alpine](#)

View a summary of image vulnerabilities and recommendations → [docker scout quickview nginx:alpine](#)

34s ~

• ▶ docker pull nginx:alpine

docker pull postgres:15-alpine

docker run -d -p 8080:80 --name temp-nginx nginx:alpine

docker exec -it temp-nginx sh

exit

docker ps

docker stop temp-nginx

docker rm temp-nginx

alpine: Pulling from library/nginx

Digest: sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913

Status: Image is up to date for nginx:alpine

docker.io/library/nginx:alpine

What's next:

View a summary of image vulnerabilities and recommendations → [docker scout quickview nginx:alpine](#)

15-alpine: Pulling from library/postgres

236e3b4c2204: Pull complete

e7426913c582: Pull complete

ecbc0f97fe87: Pull complete

d30adf595bd0: Pull complete

2521846a4c4b: Pull complete

4989bdaa0ad3: Pull complete

26065aa66f8d: Pull complete

a5b616744b9f: Pull complete

0141deae452e: Pull complete

516ac23b2d22: Pull complete

f67b1fb6e304: Download complete

f9b271aad813: Download complete

Digest: sha256:fe0737ba566a2c5b2a28f34433c0a423261900ec17b9bf7ad115e1aae7e57f1b

Status: Downloaded newer image for postgres:15-alpine

docker.io/library/postgres:15-alpine

What's next:

View a summary of image vulnerabilities and recommendations → [docker scout quickview postgres:15-alpine](#)

4be3aa31f4b23bf9cfc52b8f35068bd82f3b89683fb6c76e5b63fa47a31c9302

/ # |

```
Run 'docker run --help' for more information
0s ~ 
x ▶ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS
4be3aa31f4b2   nginx:alpine   "/docker-entrypoint..." About a minute ago Up About a minute
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   temp-nginx
0s ~ 
• ▶ docker stop temp-nginx
docker rm temp-nginx
temp-nginx
temp-nginx
0s ~ 
• ▶ |
```

Q4. What is the security benefit of using a custom tag (e.g. :alpine) rather than a standard image tag?

The :alpine variant is built on Alpine Linux (musl libc + BusyBox, roughly 5–8 MB base) instead of a Debian base (roughly 75–130 MB). The security benefit is attack surface reduction, and it is concrete rather than cosmetic:

Fewer packages means fewer CVEs. Every shipped library is a potential inherited vulnerability. The Debian-based nginx image carries a full coreutils, perl, glibc, and a package manager's worth of dependencies that nginx never calls; each one is an entry in the vulnerability scanner's output that must be triaged and patched.

Fewer post-exploitation tools. After breaking into a container, an attacker looks for what is already present - curl, wget, python, gcc, apt. Alpine's BusyBox provides stripped-down applets and omits most of these, so downloading a second-stage payload or compiling a kernel exploit in place becomes materially harder.

Smaller images pull faster and are quicker to rebuild, which lowers the friction of patching - the practical bottleneck in real vulnerability management.

The caveat worth stating: :alpine is still a floating tag. It moves as new versions are published, so it guarantees a small base but not a specific one. Production practice is to pin by digest (nginx@sha256:...) so the deployed bits are reproducible.

Task 3: Secure Image Construction (Least Privilege)

a,

Files created

[app.js](#)

```
setInterval(() => console.log("Service running..."), 5000);
```

.Dockerfile

```
FROM node:alpine
```

```
WORKDIR /app
```

COPY app.js .
USER node
CMD ["node", "app.js"]

b,

```
comp ngrok
0s ~   /Praneesh/Projects/sse-lab6-docker
• ▶ cd ~/Praneesh/Projects/sse-lab6-docker
0s ~   /Praneesh/Projects/sse-lab6-docker
• ▶ docker build -t secure-node .
[+] Building 54.7s (9/9) FINISHED
⇒ [internal] load build definition from Dockerfile
⇒ ⇒ transferring dockerfile: 114B
⇒ [internal] load metadata for docker.io/library/node:alpine
⇒ [auth] library/node:pull token for registry-1.docker.io
⇒ [internal] load .dockerignore
⇒ ⇒ transferring context: 2B
⇒ [1/3] FROM docker.io/library/node:alpine@sha256:2d984a15c9b54fd0aeb608b8e0d0d83529eb34d2966db27a1fb4f1edc3d298a3
⇒ ⇒ resolve docker.io/library/node:alpine@sha256:2d984a15c9b54fd0aeb608b8e0d0d83529eb34d2966db27a1fb4f1edc3d298a3
⇒ ⇒ sha256:80d39da31aaab43d6b63da0ed87ad747290862bd97f8901fe45eefb469cfcb0c 451B / 451B
⇒ ⇒ sha256:9098cc82c400bbe83569e25f937227d53aad39a688142b5dbb979dab5cfc5414 59.76MB / 59.76MB
⇒ ⇒ extracting sha256:9098cc82c400bbe83569e25f937227d53aad39a688142b5dbb979dab5cfc5414
⇒ ⇒ extracting sha256:80d39da31aaab43d6b63da0ed87ad747290862bd97f8901fe45eefb469cfcb0c
⇒ [internal] load build context
⇒ ⇒ transferring context: 93B
⇒ [2/3] WORKDIR /app
⇒ [3/3] COPY app.js .
⇒ exporting to image
⇒ ⇒ exporting layers
⇒ ⇒ exporting manifest sha256:f7ad531764c94da17917806e248fbd9f7392fb329d419d9542ad90baf6a407c0
⇒ ⇒ exporting config sha256:b0991f366cd3aadcd1d649cbb333dc336713ccf2f704300ecbc18eeba5dd0139e
⇒ ⇒ exporting attestation manifest sha256:8bdd7e00db988e6a7c6ca7266493808a443c52d2e17f45cac8cc661ae3ff58eb
⇒ ⇒ exporting manifest list sha256:cb44b6b71a1f8c0f841ba9a4a1e600f03853b12c6c1b655e3d9545b9d6e90225
⇒ ⇒ naming to docker.io/library/secure-node:latest
⇒ ⇒ unpacking to docker.io/library/secure-node:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cfmq01js05k4xhkwpmxw0nw1c
59s ~   /Praneesh/Projects/sse-lab6-docker
• ▶ |
```

```
0s ~ /Praneesh/Projects/sse-lab6-docker
x ▶ docker build -t secure-node .
[+] Building 5.5s (9/9) FINISHED
⇒ [internal] load build definition from Dockerfile
⇒ ⇒ transferring dockerfile: 114B
⇒ [internal] load metadata for docker.io/library/node:alpine
⇒ [auth] library/node:pull token for registry-1.docker.io
⇒ [internal] load .dockerignore
⇒ ⇒ transferring context: 2B
⇒ [1/3] FROM docker.io/library/node:alpine@sha256:2d984a15c9b54fd0aeb608b8e0d0d83529eb34d2966db27a1fb4f1edc3d298a3
⇒ ⇒ resolve docker.io/library/node:alpine@sha256:2d984a15c9b54fd0aeb608b8e0d0d83529eb34d2966db27a1fb4f1edc3d298a3
⇒ [internal] load build context
⇒ ⇒ transferring context: 27B
⇒ CACHED [2/3] WORKDIR /app
⇒ CACHED [3/3] COPY app.js .
⇒ exporting to image
⇒ ⇒ exporting layers
⇒ ⇒ exporting manifest sha256:f7ad531764c94da17917806e248fbd9f7392fb329d419d9542ad90baf6a407c0
⇒ ⇒ exporting config sha256:b0991f366cd3aadcd1d649cbb333dc336713ccf2f704300ecbc18eeba5dd0139e
⇒ ⇒ exporting attestation manifest sha256:ba5d2650eaa37c8d017bcaaf802e9f12a2adf67f492c20b99b8121297947b871
⇒ ⇒ exporting manifest list sha256:554a7ac5cb709dcd6bb8dc7968b320dd4425105088f7bf62ce847f66a7da1bca
⇒ ⇒ naming to docker.io/library/secure-node:latest
⇒ ⇒ unpacking to docker.io/library/secure-node:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/dvykd95gdvihaff7z7bms45m3
5s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker run -d --name node-test secure-node
07e0186b1c80f1064b8ed168b47199b4626ecddd8b84ffd8354ec445d4910f61
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker exec node-test whoami
node
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ |
```

C,

```
node
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker exec node-test apk add curl
ERROR: Unable to open log: Permission denied
0s ~ /Praneesh/Projects/sse-lab6-docker
x ▶ |
```

Q5. If an attacker finds a vulnerability in the Node.js application that allows arbitrary command execution, how does running as the node user limit what they can do? Arbitrary command execution gives the attacker the privileges of the compromised process, so dropping those privileges directly bounds the blast radius:

No package installation, no tooling. As shown above, apk add fails. The attacker cannot pull in curl, nmap, or a reverse-shell toolkit through the package manager and must instead smuggle in a statically-linked binary and find a writable, executable path - a far higher bar.

No tampering with the application or the OS. /app, /usr/bin, /etc, and the system libraries are root-owned. The attacker cannot backdoor app.js, replace a binary on PATH, or install a persistence mechanism such as a cron entry or a modified init script.

No binding privileged ports. Ports below 1024 require CAP_NET_BIND_SERVICE, so the attacker cannot stand up a rogue listener on 80 or 443 to impersonate a legitimate service.

Container-escape paths are largely closed. The serious escapes - abusing a mounted Docker socket, CAP_SYS_ADMIN, writing to /proc/sys/* or release_agent, loading kernel modules - all require root inside the container. A non-root process reaching the kernel attack surface first has to chain a local privilege-escalation bug on top of the RCE.

The residual risk should be stated honestly: the attacker still fully controls the application process. They can read whatever the app can read (environment variables, secrets, the database connection string), reach every network destination the container can reach, exfiltrate application data, and run in-memory payloads through the Node runtime itself. USER node prevents privilege escalation and persistence - it does not prevent data theft. It is one layer, and it is the layer that stops an application compromise from becoming a host compromise.

Task 4: Secure Configuration (Resource Limits)

a

```
0s ~ /Praneesh/Projects/sse-lab6-docker
x ▶ docker run -d --name limited-nginx --memory="256m" --cpus="0.5" nginx:alpine + docker stats --no-stream
f41393267a1bd68318c984917b78f0b31bf0ba8324d0d8131b7b7b32dbea8798
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O      PIDS
07e0186b1c80   node-test 0.00%     17.75MiB / 3.647GiB  0.48%     1.21kB / 126B   6.35MB / 0B    7
1s ~ /Praneesh/Projects/sse-lab6-docker
```

b,

```
0s ~ /Praneesh/Projects/sse-lab6-docker
x ▶ docker stop limited-nginx
limited-nginx
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker rm limited-nginx
limited-nginx
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ |
```

Q6. How do CPU and memory limits mitigate the impact of a compromised container? Both flags translate into Linux cgroup v2 controls (memory.max, cpu.max) enforced by the kernel, which converts a host-wide failure into a container-local one:

They contain denial of service. Without limits, one container shares the host's entire CPU and RAM with every other container and with the host's own daemons. A memory leak, a fork bomb, or an attacker-triggered allocation exhausts host memory and

invokes the global OOM killer, which may terminate unrelated workloads or destabilise the host. With `--memory=256m`, the cgroup's own OOM killer fires first and kills only processes inside that container. The failure stays inside the blast radius.

They cap resource-abuse payloads. Cryptomining is the most common post-exploitation payload for an exposed container; `--cpus="0.5"` caps the stolen compute at half a core, which both limits the attacker's profit and makes the anomaly stand out against the container's normal CPU profile.

They preserve neighbours. Under CPU pressure the kernel throttles the offending cgroup rather than starving co-located containers, so a compromise of one service does not degrade the others sharing the host.

They give monitoring a baseline. A declared limit is a stated expectation. Sustained operation at the ceiling is a detectable signal; without a limit there is no threshold to alert on.

The limitation worth naming: these are resource controls, not isolation controls. They do nothing against data exfiltration, lateral movement, or container escape - those need USER, capability drops, seccomp, and network policy. Resource limits address availability only, which is exactly one third of the CIA triad.

Task 5: Secure Configuration (Immutable Filesystems)

A,

```
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker run -d --name readonly-container --read-only alpine sleep 3600
e7a85d5b66e67bb555b4e0ac819bc8fedc89697b89023958a3f12839fe02a70a
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ |
```

B,

```
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker exec readonly-container sh -c 'touch /malware.sh'
touch: /malware.sh: Read-only file system
0s ~ /Praneesh/Projects/sse-lab6-docker
x ▶ |
```

C,

```
0s ~ /Praneesh/Projects/sse-lab6-docker
x ▶ docker stop readonly-container
readonly-container
3s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker rm readonly-container
readonly-container
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ |
```

Q7. If an application inherently requires write access (temporary caches or logs), how can this be accommodated securely while maintaining a read-only root filesystem? Grant writability at a specific mount point rather than lifting the read-only root. Three mechanisms, in order of preference:

--tmpfs for ephemeral data. `docker run --read-only --tmpfs`

`/tmp:rw,noexec,nosuid,size=64m` mounts a RAM-backed filesystem at `/tmp`. It is writable, it never touches disk, it vanishes when the container stops, and - crucially - `noexec` means a payload dropped there cannot be executed, while `nosuid` blocks `setuid` escalation. This is the correct choice for scratch space, PID files, and caches.

A named volume for data that must persist. `-v app_cache:/var/cache/app` writes to Docker-managed storage outside the union filesystem, so the data survives container replacement. Mount it at the narrowest path that works, and add `:ro` to any mount the application only reads from.

Do not write logs to the filesystem at all. Write to `stdout/stderr` and let the container runtime's logging driver collect them. This is the twelve-factor convention and it removes the write requirement instead of accommodating it.

The security property preserved throughout: the writable regions are enumerated, minimal, and non-executable. An attacker retains a place to put a file but no place to put a file and run it, and the application binaries, libraries, and configuration under `/usr`, `/etc`, and `/app` remain immutable. This also makes drift detection trivial - if anything outside the declared mounts has changed, the container has been tampered with.

Task 6: Multi-Container Orchestration

Compose.yml

```
readonly-container
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ cat compose.yaml
services:
  webapp:
    build: .
    ports:
      - "8080:8080"
    networks:
      - portal_net

  database:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: portal_user
      POSTGRES_PASSWORD: labpassword
      POSTGRES_DB: portal_db
    networks:
      - portal_net
    volumes:
      - db_data:/var/lib/postgresql/data

networks:
  portal_net:
    driver: bridge

volumes:
  db_data:
```

Q8. What is the network security advantage of placing the web application and database on a custom bridge network?

A user-defined bridge network is a separate Linux bridge with its own subnet, and it differs from the default bridge network in ways that matter for security:

Segmentation. Only containers explicitly attached to `portal_net` can reach each other. Every other container on the host - including anything an attacker manages to start - sits on a different bridge and cannot route to the database at all. This is network segmentation enforced at the host, and it is the foundation of least-privilege networking. No accidental exposure of the database. database publishes no host port. It is reachable only from inside `portal_net`, so the only path to Postgres from outside the host is through the web application. Compromising the app becomes a prerequisite for touching the database. On the legacy default bridge, the older `--link` pattern and the shared namespace made this separation far weaker.

DNS-based service discovery. Docker's embedded DNS resolver on a user-defined network resolves database to the service's current IP. The application never hardcodes an address, so IPs can change on restart without editing configuration - and, unlike legacy links, environment variables containing connection details are not injected into every container on the network.

Containment of lateral movement. If the web application is compromised, the attacker's reachable surface is the portal_net subnet only. They cannot scan or pivot to services on other networks. Adding a second network later (a public-facing one for webapp, a private internal one for database, the latter marked internal: true to strip its egress route) extends the same principle further.

Task 7: Resource Deployment and Cleanup

A,

```
0s ~ /Praneesh/Projects/sse-lab6-docker
• docker compose up -d --build
[+] Building 2.5s (11/11) FINISHED
⇒ [internal] load local bake definitions
⇒ reading from stdin 564B
⇒ [internal] load build definition from Dockerfile
⇒ transferring dockerfile: 114B
⇒ [internal] load metadata for docker.io/library/node:alpine
⇒ [auth] library/node:pull token for registry-1.docker.io
⇒ [internal] load .dockerignore
⇒ transferring context: 2B
⇒ [1/3] FROM docker.io/library/node:alpine@sha256:2d984a15c9b54fd0aeb608b8e0d0d83529eb34d2966db27a1fb4f1edc3d298a3
⇒ resolve docker.io/library/node:alpine@sha256:2d984a15c9b54fd0aeb608b8e0d0d83529eb34d2966db27a1fb4f1edc3d298a3
⇒ [internal] load build context
⇒ transferring context: 27B
⇒ CACHED [2/3] WORKDIR /app
⇒ CACHED [3/3] COPY app.js .
⇒ exporting to image
⇒ exporting layers
⇒ exporting manifest sha256:fee251c8d6106158ff4d4648592196e44b53b752dfcf7e63194f882e4735d0d6
⇒ exporting config sha256:07ff4eb5591de660f7dc87d19904a2451dcb449731355b7f221ebd9fc75154a6
⇒ exporting attestation manifest sha256:a13e11e257c39aa99195498612bc3a8c32f56580624b20f3d30a9d61b687aa1a
⇒ exporting manifest list sha256:e60fb4abde9d23aba53796ccfea7bf4b171471519962d0e8db1064b4b61fe51d
⇒ naming to docker.io/library/sse-lab6-docker-webapp:latest
⇒ unpacking to docker.io/library/sse-lab6-docker-webapp:latest
⇒ resolving provenance for metadata file
[+] up 5/5
✓ Image sse-lab6-docker-webapp Built
✓ Volume sse-lab6-docker_db_data Created
✓ Network sse-lab6-docker_portal_net Created
✓ Container sse-lab6-docker-database-1 Started
✓ Container sse-lab6-docker-webapp-1 Started
3s ~ /Praneesh/Projects/sse-lab6-docker
```

B,

```
3s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker compose exec webapp ping -c 3 database
PING database (172.25.0.2): 56 data bytes
64 bytes from 172.25.0.2: seq=0 ttl=42 time=0.065 ms
64 bytes from 172.25.0.2: seq=1 ttl=42 time=0.137 ms
64 bytes from 172.25.0.2: seq=2 ttl=42 time=0.117 ms

--- database ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.065/0.106/0.137 ms
2s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ |
```

C,

```
2s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker compose down --remove-orphans
[+] down 3/3
✓ Container sse-lab6-docker-webapp-1 Removed
✓ Container sse-lab6-docker-database-1 Removed
✓ Network sse-lab6-docker_portal_net Removed
4s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ |
```

Q9. Under what circumstances does an "orphaned" container occur during Compose operations?

An orphaned container is one that carries the label for the current Compose project but no longer corresponds to any service defined in the Compose file being applied.

Compose identifies its resources by the `com.docker.compose.project` label (defaulting to the directory name), so a container becomes orphaned when the file changes underneath a running stack:

A service is removed or renamed in `compose.yaml` while its container is still running. Removing a cache service leaves `<project>-cache-1` running, labelled with the project but matching no service definition.

A different Compose file is applied to the same project. Running `docker compose -f compose.prod.yaml` up in the same directory reuses the project name, so every container from the previous file that is absent from the new one is orphaned.

The project name collides. Two different stacks in identically-named directories (or both using `-p portal`) share a project namespace, and each up orphans the other's containers.

A stack is torn down with a file that no longer lists everything it created, leaving containers behind that a later up then reports as orphans. Compose warns rather than deleting them, because it cannot know whether the container is stale or deliberately kept. --remove-orphans is the explicit instruction to delete them.

The security relevance: an orphaned container is a forgotten, unpatched, still-running workload. It keeps its network attachments and any published ports, it stops appearing in the deployment's inventory, and it stops being covered by the patch cycle - a textbook shadow asset. Cleaning them up is asset-inventory hygiene, not just tidiness.

Task 8: Vulnerability Assessment

```
4s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker scout cves nginx:alpine
  ✓ SBOM obtained from attestation, 87 packages found
  ✓ Provenance obtained from attestation
  ✗ Detected 4 vulnerable packages with a total of 18 vulnerabilities

## Overview



|                 | Analyzed Image                                                                                           |
|-----------------|----------------------------------------------------------------------------------------------------------|
| Target          | nginx:alpine                                                                                             |
| digest          | 1f25fedd50ae                                                                                             |
| platform        | linux/amd64                                                                                              |
| provenance      | https://github.com/nginx/docker-nginx.git<br>b8590bd36b4504b9b847fcf2e98a9111dcae85fa                    |
| vulnerabilities | 1C 9H 3M 3L 2?                                                                                           |
| size            | 26 MB                                                                                                    |
| packages        | 87                                                                                                       |
| Base image      | oisupport/staging-amd64:33e33524cfc21d9956b88c4c135ef946fa92f8724f1505c31e6569e3def4946e<br>4bfe6239cfb9 |



## Packages and Vulnerabilities

1C 7H 1M 0L 1? openssl 3.5.7-r0
pkg:apk/alpine/openssl@3.5.7-r0?os_name=alpine&os_version=3.24

mainline/alpine/Dockerfile (0:0)

✗ CRITICAL CVE-2026-63073
https://scout.docker.com/v/CVE-2026-63073
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0

✗ HIGH CVE-2026-63076
https://scout.docker.com/v/CVE-2026-63076
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0

✗ HIGH CVE-2026-63075
https://scout.docker.com/v/CVE-2026-63075
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0
```

- × **HIGH** CVE-2026-63072
<https://scout.docker.com/v/CVE-2026-63072>
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0
- × **HIGH** CVE-2026-54874
<https://scout.docker.com/v/CVE-2026-54874>
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0
- × **HIGH** CVE-2026-18798
<https://scout.docker.com/v/CVE-2026-18798>
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0
- × **HIGH** CVE-2026-14457
<https://scout.docker.com/v/CVE-2026-14457>
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0
- × **HIGH** CVE-2026-14456
<https://scout.docker.com/v/CVE-2026-14456>
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0
- × **MEDIUM** CVE-2026-63074
<https://scout.docker.com/v/CVE-2026-63074>
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0
- × **UNSPECIFIED** CVE-2026-75803
<https://scout.docker.com/v/CVE-2026-75803>
Affected range : <3.5.8-r0
Fixed version : 3.5.8-r0

0C 2H 2M 0L expat 2.8.2-r0
pkg:apk/alpine/expat@2.8.2-r0?os_name=alpine&os_version=3.24

```
&& apk add --no-cache curl ca-certificates
```

× **HIGH** CVE-2026-76641

<https://scout.docker.com/v/CVE-2026-76641>

Affected range : <2.8.4-r0

Fixed version : 2.8.4-r0

× **HIGH** CVE-2026-66046

<https://scout.docker.com/v/CVE-2026-66046>

Affected range : <2.8.4-r0

Fixed version : 2.8.4-r0

× **MEDIUM** CVE-2026-76956

<https://scout.docker.com/v/CVE-2026-76956>

Affected range : <2.8.4-r0

Fixed version : 2.8.4-r0

× **MEDIUM** CVE-2026-76957

<https://scout.docker.com/v/CVE-2026-76957>

Affected range : <2.8.4-r0

Fixed version : 2.8.4-r0

0C

0H

0M

3L

libxml2 2.13.9-r2

apk:apk/alpine/libxml2-2.13.9-r2306.name=alpineos-version=3.26

× **LOW** CVE-2026-0989

<https://scout.docker.com/v/CVE-2026-0989>

Affected range : ≤2.13.9-r2

Fixed version : not fixed

× **LOW** CVE-2025-8732

<https://scout.docker.com/v/CVE-2025-8732>

Affected range : ≤2.13.9-r2

Fixed version : not fixed

× **LOW** CVE-2026-11979

<https://scout.docker.com/v/CVE-2026-11979>

Affected range : ≤2.13.9-r2

Fixed version : not fixed

0C

0H

0M

0L

1? nghttp2 1.69.0-r0

```
18 vulnerabilities found in 4 packages
CRITICAL 1
HIGH 9
MEDIUM 3
LOW 3
UNSPECIFIED 2

What's next:
View base image update recommendations → docker scout recommendations nginx:alpine

16s ~ /Praneesh/Projects/sse-lab6-docker
```

Q10. Why is continuous vulnerability scanning necessary even if an image was deemed secure at the time of its initial build?

Because "secure" is a statement about a moment, not a property of the artifact. The image is immutable; the threat landscape is not.

New CVEs are disclosed against unchanged code. A vulnerability existing in OpenSSL today may not be published until months from now. The scan on build day returned clean because the CVE did not yet exist in any database - not because the flaw was absent. The image is exactly as vulnerable as it always was; only the world's knowledge changed.

Vulnerability databases lag and get corrected. NVD entries are amended, severities are re-scored, and affected-version ranges are widened after the fact. A package once judged unaffected is later marked vulnerable.

Long-lived deployments drift far from their build date. A container image built once and run for a year accumulates every CVE disclosed in that year. Without rescanning, the gap between actual and believed exposure widens continuously - and the image cannot patch itself, since immutability means the only remedy is rebuild and redeploy.

Scanner coverage improves. Tools add ecosystems, improve package detection, and fix false negatives. Rescanning the same image with a newer scanner routinely surfaces findings the original run missed entirely.

Exploit maturity changes the risk, not just the score. A theoretical medium-severity bug becomes urgent the day a working exploit is published or it appears in CISA's KEV catalog. Continuous scanning with threat intelligence is what turns a static severity into an actionable priority.

This is why vulnerability management is defined as a cycle - discover, prioritise, remediate, verify, repeat - and why CI/CD pipelines gate on scans at build time and registries rescan stored images on a schedule. A point-in-time scan answers "is this image known to be vulnerable today?"; only continuous scanning keeps that answer true tomorrow.

Cleanup

```
16s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ docker compose down --remove-orphans -v
  docker rm -f node-test
  docker rmi secure-node
[+] down 1/1
✓ Volume sse-lab6-docker_db_data Removed
node-test
Untagged: secure-node:latest
Deleted: sha256:554a7ac5cb709dcd6bb8dc7968b320dd4425105088f7bf62ce847f66a7da1bca
0s ~ /Praneesh/Projects/sse-lab6-docker
• ▶ |
```